

Target Platform: macOS 14.0+ (Sonoma / Sequoia)	Language: Swift 5.10+	Editor / Build: VS Code + Swift Package Manager (CLI)
Persistence: SwiftData & Disk Cache	Metadata Source: Jikan API v4 (MyAnimeList)	Architecture: Local-First MVVM + Service Layer

1. Executive Summary & Philosophy

Chibiori is a high-performance, native macOS application designed strictly for local anime tracking. Unlike cloud-synced applications, Chibiori operates under a **Local-First, Absolute User Sovereignty** model. MyAnimeList (MAL) metadata is consumed purely as an information source via the Jikan v4 REST API; list management, rating history, notes, and progress timestamps remain 100% stored on the user's Mac.

Core Architectural Promise: Zero dependency on MAL account authentication or write access. The user maintains immutable control over queue order, dates added, and custom status transitions without server side flattening.

2. Complete Tech Stack Overview

Layer / Component	Technology Selected	Rationale & Architecture Role
Target OS	macOS 14.0+ (Sonoma)	Leverages modern SwiftData macros, updated SwiftUI NavigationSplitView APIs, and native macOS desktop controls.
UI Framework	SwiftUI	Declarative, lightweight rendering tailored for macOS window layout, toolbar items, inspectors, and shortcut integration.
IDE & CLI Tools	VS Code + Xcode CLT + SPM	Full terminal-driven development using Swift Package Manager, SourceKit-LSP, and CodeLLDB plugin in VS Code.
Local Database	SwiftData Framework	Provides native object persistence with zero core-data boilerplate, automatic schema migrations, and high-performance `@Query` bindings.
Networking Layer	Native URLSession + Async/Await	Clean, zero-dependency async network client wrapped in Swift Actors to ensure thread-safe operation.
Metadata API	Jikan API v4 (REST)	Keyless, high-reliability REST API proxying MyAnimeList data without requiring developer key registration.
Image Management	Local Disk Storage + NSImage	Fetches poster art and writes to Application Support/Chibiori/Covers for complete offline accessibility.
Data Backup	Structured JSON Schema	Full single-file JSON import and export controller for manual backups and migration across machines.

3. Data Schema & SwiftData Models

The persistence architecture isolates local user interactions from external MAL metadata snapshots to guarantee data consistency.

3.1 Model Definition: TrackedAnime

```
import SwiftData
import Foundation

@Model
final class TrackedAnime {
    // Unique Primary Key mapping to MyAnimeList ID
    @Attribute(.unique) var malID: Int

    // External MAL Metadata Snapshot (Cached locally)
    var title: String
    var japaneseTitle: String?
    var synopsis: String
    var coverImageFilename: String? // Relative filename in Local Storage
    var coverImageRemoteURL: String
    var airingStatusRaw: String      // "Currently Airing", "Finished Airing", "Not Yet Aired"
    var totalEpisodes: Int?          // Nil for unknown / ongoing
    var broadcastDayRaw: String?     // e.g. "Mondays"
    var broadcastTimeUTC: String?    // e.g. "15:00"
    var malScore: Double?
    var seasonYear: String?          // e.g. "Spring 2026"
    var genres: [String]             // Genre tags array

    // User Local State (Immutable / User Controlled)
    var watchStatusRaw: String       // "planToWatch", "watching", "completed", "onHold",
    "dropped"
    var currentEpisodeProgress: Int
    var userRating: Int?             // 1 to 10
    var personalNotes: String

    // Timestamps
    var dateAdded: Date              // Refreshed when re-added to queue
    var dateStarted: Date?           // Recorded on shift to "watching"
    var dateCompleted: Date?         // Recorded on shift to "completed"
    var statusLastUpdatedAt: Date    // Recorded on any status shift

    init(malID: Int, title: String, synopsis: String, coverImageRemoteURL: String,
    airingStatusRaw: String) {
        self.malID = malID
        self.title = title
        self.synopsis = synopsis
        self.coverImageRemoteURL = coverImageRemoteURL
        self.airingStatusRaw = airingStatusRaw
        self.watchStatusRaw = WatchStatus.planToWatch.rawValue
        self.currentEpisodeProgress = 0
        self.personalNotes = ""
        self.genres = []
        self.dateAdded = Date()
        self.statusLastUpdatedAt = Date()
    }
}
```

3.2 Local Enums

Enum Type	Cases	Behavior & Usage
WatchStatus	.planToWatch, .watching, .completed, .onHold, .dropped	Determines sidebar grouping. Transitioning updates statusLastUpdatedAt and updates start/complete dates.

Enum Type	Cases	Behavior & Usage
AiringStatus	.currentlyAiring, .finishedAiring, .notYetAired	Parsed from Jikan response; used for hybrid filtering inside local watchlists.

4. Queue Ordering & Re-Add Mechanics

Standard MAL sync resets or locks historical addition dates. Chibiori solves this through explicit local timestamp manipulation:

- 1. Default Queue Sort:** The *Plan to Watch* view applies a primary sort descriptor of `SortDescriptor(\TrackedAnime.dateAdded, order: .reverse)`.
- 2. Re-Addition Behavior:** If an anime is removed from *Plan to Watch* and subsequently re-added, or if the user explicitly triggers a "Re-queue" action, `dateAdded` is overwritten with `Date.now`.
- 3. Queue Position:** The updated `dateAdded` timestamp naturally elevates the entry to the top of the queue without altering other metadata.
- 4. Preservation:** Historical start dates (`dateStarted`) and completion dates (`dateCompleted`) are preserved independently of `dateAdded`.

5. Networking & Rate Limiting Architecture

Jikan API v4 enforces a threshold of **3 requests per second** (~60 requests/minute). To ensure Chibiori never triggers HTTP 429 rate-limit responses, networking is routed through an isolated Swift Actor:

```
actor JikanAPIService {
    private var lastRequestTime: Date = .distantPast
    private let minimumInterval: TimeInterval = 0.35 // 350ms throttle (~2.8 req/sec max)

    func fetchSearchResults(query: String) async throws -> [JikanAnimeDTO] {
        try await throttle()
        let encodedQuery =
query.addingPercentEncoding(withAllowedCharacters: .urlQueryAllowed) ?? ""
        let url = URL(string: "https://api.jikan.moe/v4/anime?q=\(encodedQuery)&sfw=true")!
        let (data, response) = try await URLSession.shared.data(from: url)

        guard let httpResponse = response as? HTTPURLResponse, httpResponse.statusCode == 200
    else {
        throw APIError.invalidResponse
    }

    let decoded = try JSONDecoder().decode(JikanSearchResponse.self, from: data)
    return decoded.data
    }

    private func throttle() async {
        let now = Date()
        let timeSinceLast = now.timeIntervalSince(lastRequestTime)
        if timeSinceLast < minimumInterval {
            let sleepTime = UInt64((minimumInterval - timeSinceLast) * 1_000_000_000)
            try? await Task.sleep(nanoseconds: sleepTime)
        }
        lastRequestTime = Date()
    }
}
```

6. UI & Navigation Architecture

Sidebar Hierarchy (NavigationSplitView)

• MY LIBRARY

All Anime

Watching

Plan to Watch

Completed

On Hold

Dropped

• EXPLORE

Discover / Search (MAL)

Weekly Calendar

• DATA

Import / Export JSON

Main Content & Controls

• Toolbar Controls:

Search local library, view mode picker (Poster Grid vs. Compact Table), Airing Status filter pill.

• Card Controls:

Inline poster art, title, episode counter with rapid +1 button.

• Detail Inspector:

Right-hand drawer displaying synopsis, studio, score, episode slider, user rating, notes editor, and timestamps.

7. Data Export & Import Specification

Chibiori supports single-file JSON backup files. Below is the strict structural schema:

```
{
  "version": "1.0",
  "exportedAt": "2026-08-13T15:30:00Z",
  "app": "Chibiori",
  "records": [
    {
      "malID": 5114,
      "title": "Fullmetal Alchemist: Brotherhood",
      "watchStatus": "completed",
      "currentEpisodeProgress": 64,
      "userRating": 10,
      "personalNotes": "Masterpiece pacing and ending.",
      "dateAdded": "2026-01-10T10:00:00Z",
      "dateStarted": "2026-01-11T12:00:00Z",
      "dateCompleted": "2026-01-25T18:30:00Z",
      "airingStatus": "Finished Airing",
      "totalEpisodes": 64,
      "malScore": 9.1,
      "coverImageRemoteURL": "https://cdn.myanimelist.net/images/anime/1208/94745.jpg"
    }
  ]
}
```

8. VS Code & Command Line Tools Setup

8.1 SPM Manifest (Package.swift)

```
// swift-tools-version: 5.10
import PackageDescription

let package = Package(
```

```

    name: "Chibiori",
    platforms: [
        .macOS(.v14)
    ],
    targets: [
        .executableTarget(
            name: "Chibiori",
            path: "Sources"
        )
    ]
}
)

```

8.2 Debug Configuration (`.vscode/launch.json`)

```

{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "lldb",
      "request": "launch",
      "name": "Debug Chibiori",
      "program": "${workspaceFolder}/.build/debug/Chibiori",
      "args": [],
      "cwd": "${workspaceFolder}",
      "preLaunchTask": "swift: Build Debug Chibiori"
    }
  ]
}

```

8.3 Development CLI Workflow & Commands

Task / Command	Terminal Syntax	Description
Initialize Project	<code>swift package init --type executable</code>	Generates executable package structure in current directory.
Build Project	<code>swift build</code>	Compiles Swift sources using macOS 14 deployment target.
Run Application	<code>swift run</code>	Compiles (if changed) and launches native macOS window executable.
Clean Build	<code>swift package clean</code>	Clears <code>.build</code> directory artifacts.

8.4 Implementation Roadmap

- Phase 1: SPM & SwiftData Setup (Day 1)** — Initialize SPM project, target macOS 14+, configure SwiftData container, and build `TrackedAnime` model.
- Phase 2: Networking Actor (Day 2)** — Implement `JikanAPIService` actor throttling and local artwork disk directory persistence.
- Phase 3: Core UI & Sidebar (Day 3–4)** — Construct `NavigationSplitView` views, poster grids, table rows, and inline `+1` episode increment controls.
- Phase 4: Discover & Schedule (Day 5)** — Build live MAL search tab, seasonal query view, and weekly schedule mapping for active shows.
- Phase 5: Backup & CodeLLDB Setup (Day 6)** — Integrate JSON Export/Import controllers and verify VS Code debugging launch tasks.